

PROJECT STATUS REPORT

# FlowOS

Virtual Queue Management Platform — “Skip the Line”

---

**Prepared for:** Daily Stand-up (Manager & Team)

**Date:** 8 June 2026

**Status:** Working end-to-end (web demo live) · native-ready

**Scope:** Architecture · Tech Stack · Modules · Screens · Database · Real-time · Roadmap · Q&A

## 1. Project Overview

### What is FlowOS?

FlowOS is a **“skip the line” platform** — a virtual queue management system. Instead of physically standing in line at a clinic, bank, salon, restaurant, or government office, customers join the queue **from their phone** and watch their turn approach in real time.

### What problem does it solve?

Waiting in physical queues wastes time and is frustrating — you can’t see how long the wait is, how many people are ahead, or when it’s your turn. Businesses also struggle to manage crowds. FlowOS makes waiting **transparent** (live position + estimated time), **remote** (join from anywhere), and **efficient** (businesses run their counters from an app).

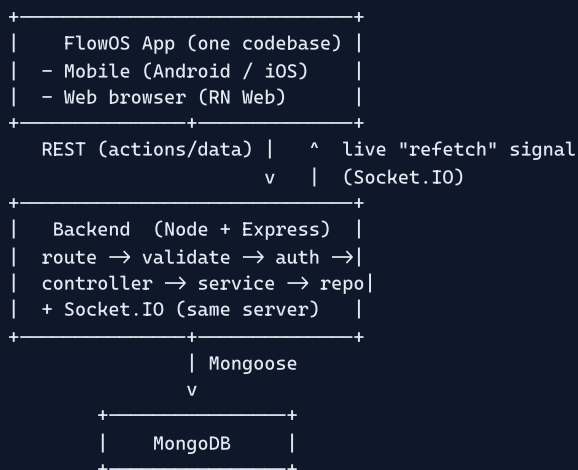
### Current project goal

Deliver a working, demonstrable end-to-end product: a business can sign up, set itself up, and serve customers; a customer can discover a business, join a queue, and track it live. This is **working today in a web browser**, talking to a real backend and database. The same code is built to become native Android/iOS apps with no rewrite.

## 2. Architecture Overview

Think of FlowOS as **three layers that talk to each other**: the app (what users see), the backend (the brain/rules), and the database (the memory). A real-time channel sits alongside to push live updates.

- **Frontend (the app)**: One React Native codebase. Runs as a **mobile app** (Android/iOS) and **in the browser** (React Native Web). Shows screens, collects input, calls the backend — and asks the backend for the truth rather than holding its own logic.
- **Backend (the brain)**: Node.js + Express, a clean **layered modular monolith**. Every request flows: `route → validate → check login → check permission → controller → service → repository → database`.
- **Database (the memory)**: MongoDB via Mongoose. Only the repository layer touches it, keeping data access clean.
- **Real-time (the live wire)**: Socket.IO. On a change, the server sends a tiny **“something changed” signal** and the app **refetches** the latest — the **“signal → fetch”** model.
- **How they communicate**: the app calls the backend over REST for all actions/data; the backend reads/writes MongoDB; live updates go out as Socket.IO signals. Push, email, and AI are plug-ins behind interfaces that degrade gracefully if unconfigured.



Plug-in services (behind interfaces):

Push (Firebase) . Email (SMTP) . AI (Groq) . File storage (local disk)

### 3. Tech Stack Used

| Technology                                                                                                                                                                                                                                                       | Why we chose it                                                 | Purpose                                         | How it connects                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|-------------------------------------------------|-----------------------------------------------------------|
| <b>TypeScript</b>                                                                                                                                                                                                                                                | Catches errors before runtime; self-documenting                 | Type safety across the project                  | Used in app + backend; shared concepts defined once       |
| <b>React Native</b>                                                                                                                                                                                                                                              | One codebase for Android + iOS + web; native feel               | Builds all user screens                         | Calls backend via Axios + Socket.IO                       |
| <b>React Native Web</b>                                                                                                                                                                                                                                          | Demo the mobile app in a browser without Android tooling        | Runs the same app in the browser today          | Maps RN components to web; same screens/API               |
| <b>Node.js</b>                                                                                                                                                                                                                                                   | One language (TS) client+server; great for live, I/O-heavy apps | Runs the backend                                | Hosts Express + Socket.IO                                 |
| <b>Express.js</b>                                                                                                                                                                                                                                                | Minimal, standard web framework                                 | Routing + request handling                      | Wires modules → controllers → services                    |
| <b>MongoDB</b>                                                                                                                                                                                                                                                   | Flexible documents fit evolving data; strong geo/search         | Stores all data                                 | Accessed only via Mongoose repositories                   |
| <b>Mongoose</b>                                                                                                                                                                                                                                                  | Structure, validation, indexes on MongoDB                       | Defines the 16 data models                      | Bridges services and MongoDB                              |
| <b>Socket.IO</b>                                                                                                                                                                                                                                                 | Reliable real-time with auto-reconnect                          | Live position/ETA, “it’s your turn”, dashboards | Same server; app subscribes to “rooms”                    |
| <b>JWT</b>                                                                                                                                                                                                                                                       | Stateless, standard, secure login                               | Keeps users logged in securely                  | Sent per request; verified by backend + socket            |
| <b>Firebase (FCM)</b>                                                                                                                                                                                                                                            | Industry standard for mobile push                               | Background “it’s your turn” alerts              | Coded behind interface; activates with a Firebase account |
| <b>Axios</b>                                                                                                                                                                                                                                                     | Clean HTTP client with interceptors                             | All backend calls from the app                  | Auto-attaches token; auto-refreshes on expiry             |
| <b>Jest (+ Supertest)</b>                                                                                                                                                                                                                                        | Standard testing framework                                      | Automated tests for backend + app               | Runs locally and in CI on every change                    |
| <b>Supporting:</b> zod (validation), bcrypt (password hashing), pino (logging), helmet/cors (security), multer (image uploads), Groq (AI), nodemailer (email) — each solves one concern, all wired through a central composition root so any one can be swapped. |                                                                 |                                                 |                                                           |

## 4. Module-by-Module Progress

16 backend modules, same layered pattern. Purpose · status · APIs · screens · collections.

| Module                              | Purpose                                     | Status                    | APIs / Screens / Collections                                                                                                                                  |
|-------------------------------------|---------------------------------------------|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Authentication</b>               | Secure sign-up/login                        | Complete                  | register, login, refresh, logout, forgot/reset password, “me” · Login/Register/Forgot screens · <code>users</code> , <code>refreshTokens</code>               |
| <b>User</b>                         | Profile + settings                          | Complete                  | get/update profile + settings, onboarding · Profile · <code>users</code>                                                                                      |
| <b>Business</b>                     | Register, setup, activate, discover, delete | Complete                  | explore (search + near-me), mine, get, create, update (+logo), delete (cascade) · Explore/Details/Dashboard/Create/Setup · <code>businesses</code>            |
| <b>Queue Management</b>             | Queues + live queue engine                  | Complete                  | CRUD queues; join/leave/list/call-next/serve/complete/no-show; my-entries · QueueForm/QueueManager/Activity · <code>queues</code> , <code>queueEntries</code> |
| <b>Notification</b>                 | Alerts + in-app feed                        | In-app done; push pending | list, mark-read, mark-all, device tokens · Notifications · <code>notifications</code> , <code>deviceTokens</code>                                             |
| <b>Analytics</b>                    | Business dashboards                         | Backend done; UI pending  | 24h summary, detailed window · (on Dashboard) · <code>analyticsEvents</code>                                                                                  |
| <b>Reviews</b>                      | Ratings & reviews                           | Backend done; UI pending  | list, create, update, delete · — · <code>reviews</code>                                                                                                       |
| <b>Appointments</b>                 | Time-slot bookings                          | Backend done; UI pending  | book, my list, update, cancel, business list · — · <code>appointments</code>                                                                                  |
| <b>Staff Management</b>             | Staff per business                          | Backend done; UI pending  | list, add by email, change role, remove · — · <code>staffMembers</code>                                                                                       |
| <b>AI Assistant</b>                 | Groq-backed helper + recommendations        | Backend done; UI pending  | chat, list/get conversations · — · <code>aiConversations</code>                                                                                               |
| <b>Support</b>                      | Help articles + tickets                     | Backend complete          | articles (public), create/list/get tickets · — · <code>supportTickets</code>                                                                                  |
| <b>+ System / Favorites / Media</b> | Health, saved places, image upload          | Complete                  | health/config; favorites · Media upload (new) · <code>savedBusinesses</code>                                                                                  |

## 5. Screen Flow

13 screens, split by role. After login, the app shows the **customer** or **business** experience automatically based on the user's role.

### Customer flow

```

Login  → Register / Forgot Password
|
v
Explore  (search + "Near me", with images)
|
v
Business Details  (cover image, info, queues)
|
v
Join Queue
|
v
Activity / "My Queues"  (LIVE position + ETA, updates by itself)
|
v
Notifications  ("It's your turn!")      + Profile (account/settings)

```

### Business owner / staff flow

```

Login / Register
|
v
Dashboard  (your businesses + status chips)
|
v
Create Business
|
v
Business Setup  (set hours, upload thumbnail, DRAFT → ACTIVE, delete)
|
v
Queue Form  (create/edit a service queue)
|
v
Queue Manager  (operate LIVE: call-next, serve, complete, no-show)
                  + Notifications . Profile

```

Key gate: a business stays a **DRAFT** (hidden) until the owner **activates** it — only then does it appear in customer Explore.

## 6. Database Design

**Collections created (16):** `users`, `businesses`, `staffMembers`, `queues`, `queueEntries`, `appointments`, `savedBusinesses`, `reviews`, `notifications`, `deviceTokens`, `analyticsEvents`, `supportTickets`, `aiConversations`, `categories`, `refreshTokens`, `kycRequests`.

### Relationships (plain terms)

- A **user** can own **businesses**, and be **staff** at businesses (role: Owner / Manager / Staff).
- A **business** has many **queues**; a **queue** has many **entries** (each entry = one customer's place in line).
- A **business** also has **reviews**, **appointments**, and **analytics events**.
- A **user** has their own entries, appointments, saved places, reviews, notifications, devices.
- A user can review or save a business **only once** (enforced by the database).

```

User <-- StaffMember >-- Business <-- Queue <-- QueueEntry >-- User
|                                     | +--> Review >-- User      (one per user+business)
|                                     | +--> Appointment
|                                     | +--> analyticsEvents
+--> Notification / Device / SavedBusiness / RefreshToken

```

**How data flows:** app request → backend validates & checks permissions → business rules run → repository reads/writes MongoDB → clean JSON answer returns. Any change others care about also fires a real-time signal.

## 7. Real-Time Flow

FlowOS uses the “**signal** → **fetch**” approach: the server sends a small nudge, and the app fetches the fresh details — fast updates, always-correct data.

1. Customer joins a queue → app opens a **Socket.IO** connection (verified with their login token).
2. App **subscribes** to rooms: their user room, the queue room, and (owners) their business room.
3. An operator taps “**Call next**” → backend updates the database (one person WAITING → CALLED).
4. Backend **emits a signal** to the queue room and affected user rooms (“queue changed”).
5. Every customer's app in that queue **refetches** position + ETA → screen updates **automatically**.
6. On the customer's turn, the backend also **writes a notification** and signals their user room → feed + live badge update instantly. (With Firebase on, this also becomes a phone push.)

**When position changes:** position = people ahead + 1; ETA = people ahead × average service time, where the average **self-tunes** from real serving times.

## 8. Work Completed So Far (by phase)

---

### Phase 1 — Foundation & Secure Login ✓

- Layered backend skeleton (app factory, boot sequence, central error handling, structured logging).
- Validated configuration that **fails fast** if secrets are missing.
- MongoDB connection + 16 data models with proper indexes (geo, text, uniqueness, auto-expiry).
- Full **authentication**: register, login, logout, refresh, password reset, "me".
- Security: hashed passwords, **refresh-token rotation + reuse detection, role-based permissions**, rate-limiting.
- Mobile: Login/Register, secure token storage, role-based routing, silent token refresh.

### Phase 2 — Core Product (two-sided platform) ✓

- All core backend modules: businesses, queues, **live queue engine**, notifications, analytics, reviews, appointments, staff, AI, support, favorites, system.
- Queue engine: full lifecycle, live position, self-tuning ETA, atomic "call-next", anti-double-join.
- **Real-time** with Socket.IO (rooms + signals) on the same server.
- Mobile core screens: Explore, Business Details, Activity (live), Dashboard, Create Business, Queue Form, Queue Manager, Notifications, Profile.
- Verification scripts (smoke + socket smoke).

### Phase 3 — Self-service completion, web, uploads & polish ✓

- **Business Setup + Activation** (hours editor, DRAFT→ACTIVE toggle, profile edits).
- **Delete Business** (owner-only, cascades cleanly).
- **Real-time on the client**: notifications feed + dashboard update live.
- **Forgot-password UI**; **device-token registration** lifecycle wired.
- **"Near me"** location search in Explore.
- **Runs in the browser** via React Native Web (demo without Android tooling).
- **Owner image uploads** (this week): upload endpoint + local storage + thumbnail picker + cross-origin display fix.
- **Hyderabad demo dataset** (this week): realistic businesses (restaurant, salon, hospitals/clinic, banks, education) — active, with real images and services.
- **Automated tests + CI**: backend (~21 tests) and mobile (~13 tests) + smoke checks, on every change via GitHub Actions.

## 9. Current Status

---

### Fully completed

Secure auth + roles; users; businesses (setup, activation, delete); queues + the live queue engine; in-app real-time notifications; analytics APIs; reviews; appointments; staff management; AI assistant (backend); support; favorites; owner image uploads; full customer + business screen flows; browser (web) build; automated tests + CI; demo data.

### Partially completed

- **Background push (Firebase)**: fully coded, needs a Firebase account to switch on.
- **Password-reset email**: logic done, emails currently print to the server console — needs an email provider (SMTP).
- **Image storage**: working, but local-disk only — move to cloud for production.
- **Secondary UIs**: Analytics, Reviews, Appointments, Staff, AI have working backends; dedicated screens pending.

### Still pending

Dedicated Analytics/Staff/Appointments/Reviews/AI screens; onboarding screen; admin tools; KYC/verification; payments; native app-store builds.

## 10. Future Roadmap

---

- **Next up:** build the secondary screens (Analytics, Reviews, Appointments, Staff, AI) on the existing APIs; add an onboarding screen.
- **Mobile app:** no rewrite — install the Android/iOS toolchain and build; only the API address differs from the web build.
- **Push notifications:** add a Firebase account + the mobile messaging client → background alerts go live (lifecycle already wired).
- **Production deployment:** host the backend, point at managed MongoDB (Atlas), set secrets, publish the web build to static hosting, submit native apps. Move image storage to cloud and emails to a real provider.
- **Scaling:** add a Socket.IO Redis adapter for multi-server live updates; the rest already scales horizontally.



## 11. Stand-Up Meeting Summary (≈2 minutes)

"FlowOS is our virtual queue — a 'skip the line' app. Customers join a queue from their phone and watch their position and wait time update live, and get notified when it's their turn. Businesses get an app to publish their venue and run their counters in real time.

The product works end-to-end **today**. A business can register, set hours, activate, create queues, and serve customers; a customer can find a business, join, and track it live — all running in a web browser against a real backend and database. It's one codebase that also powers native iPhone/Android apps, which are ready to build.

Under the hood: a clean Node/Express backend with 16 modules, MongoDB for storage, and Socket.IO for live updates. The hard parts — the real-time queue engine, the two-sided flow, secure login with roles, and the cross-platform foundation — are **built and tested**, with automated tests on every change.

This week we added **owner image uploads** and a realistic **Hyderabad demo dataset** for showcasing.

What's left before launch is mostly **external hookups** — push notifications (needs a Firebase account) and password-reset emails (needs an email provider) — plus a few secondary screens whose backends are already done. No architectural risks remain; the path to launch is short and well understood."

## 12. Manager Questions — Preparation (Top 20)

---

### 1. Is the product working end-to-end?

Yes — register → activate business → create queues → join → live tracking → serve, all working in the browser against a real backend/DB.

### 2. Can I see a demo?

Yes — the web build runs locally; we have a Hyderabad demo dataset with real businesses, services, and images.

### 3. Is it mobile or web?

One React Native codebase. Runs in the browser now (for demos) and is ready to build as native Android/iOS — same code, not a rewrite.

### 4. Why MongoDB instead of SQL?

Our data is flexible/nested (hours, settings, location) and needs strong geo/search; MongoDB fits and lets us iterate fast.

### 5. How does the live updating work?

Socket.IO sends a “changed” signal; the app refetches the latest from the backend — fast and always accurate.

### 6. Is login secure?

Yes — hashed passwords, JWT with refresh-token rotation and reuse detection, role-based permissions, rate-limiting.

### 7. What roles exist?

Customer, Business Owner, Staff, Platform Admin (admin role exists; admin UI is future work).

### 8. What's finished vs. pending?

Finished: all core backends + customer/business screens + real-time + tests. Pending: secondary screens and external hookups.

### 9. Are notifications working?

In-app real-time: yes. Background phone push: coded, needs a Firebase account to switch on.

### 10. Do password-reset emails work?

Logic works; emails currently log to the console — we need an email provider to send real emails.

### 11. How accurate is the wait-time estimate?

People-ahead × average service time, and the average self-tunes from real serving times, so it improves with use.

### 12. Is the code tested?

Yes — backend (~21) and mobile (~13) automated tests plus smoke checks, run automatically on every change (CI).

### 13. How many features/modules are built?

16 backend modules and 16 database collections; 13 app screens.

### 14. What if two customers are called at once?

The “call-next” operation is atomic in the database, so positions stay consistent — no double-serving.

### 15. Can a business be hidden until ready?

Yes — businesses are DRAFT (hidden) until the owner activates them; only active ones show in Explore.

### 16. Can businesses upload their own images?

Yes — added this week (owner thumbnail upload). Local storage for now; cloud for production.

### 17. What does launching the mobile apps take?

Install the build toolchain, build, point to the live API, add Firebase for push, submit to the stores.

### 18. Will it scale to many users?

Yes — the backend is stateless and scales horizontally; only change for multi-server is a Socket.IO Redis adapter.

### 19. Biggest risks or gaps?

No architectural risks; remaining items are configuration/finishing (push, email, cloud storage, secondary screens).

### 20. What's the next milestone?

Build the secondary screens on the existing APIs, then wire up push + email, then prepare production deployment.